

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente División de Ciencias de la Ingeniería
Organización de Lenguajes y Compiladores 2
Yenifer
Sección: A



Manual de Usuario

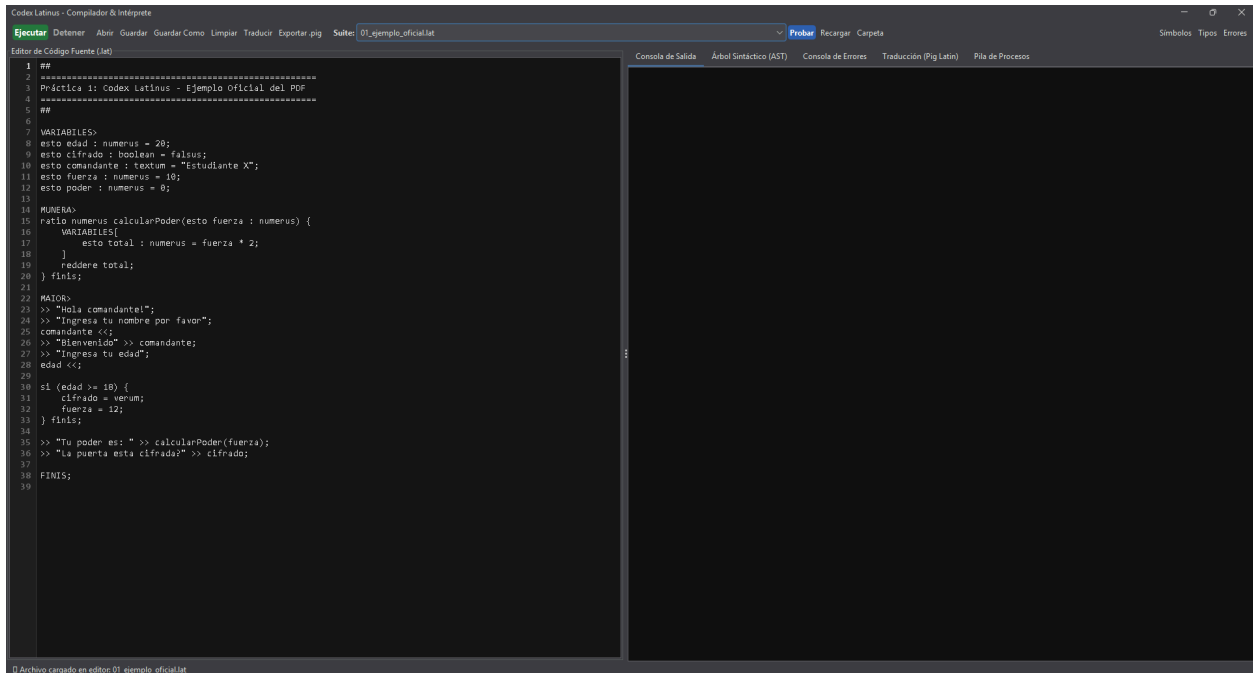
Carlos Raúl Alberto López Peláez 202031871

Quetzaltenango, 24 de Agosto del 2026

Índice

1. Inicio de la Aplicación.....	3
1.1. Ejecutar.....	4
1.2. Detener.....	4
2. Apartados de la aplicación.....	5
2.1. Consola.....	5
2.2. Árbol Sintáctico.....	6
2.3. Consola de errores.....	7
2.4. Traducción Pig Latin.....	7
2.5. Pila de procesos.....	8
3. Barra de opciones.....	9
3.1. Abrir.....	9
3.2. Guardar/Guardar Como.....	9
3.3. Limpiar.....	11
3.4. Traducir.....	11
3.3. Exportar.....	12
3.3. Suite.....	12
3.3. Probar/Recargar.....	13
3.3. Carpeta.....	14

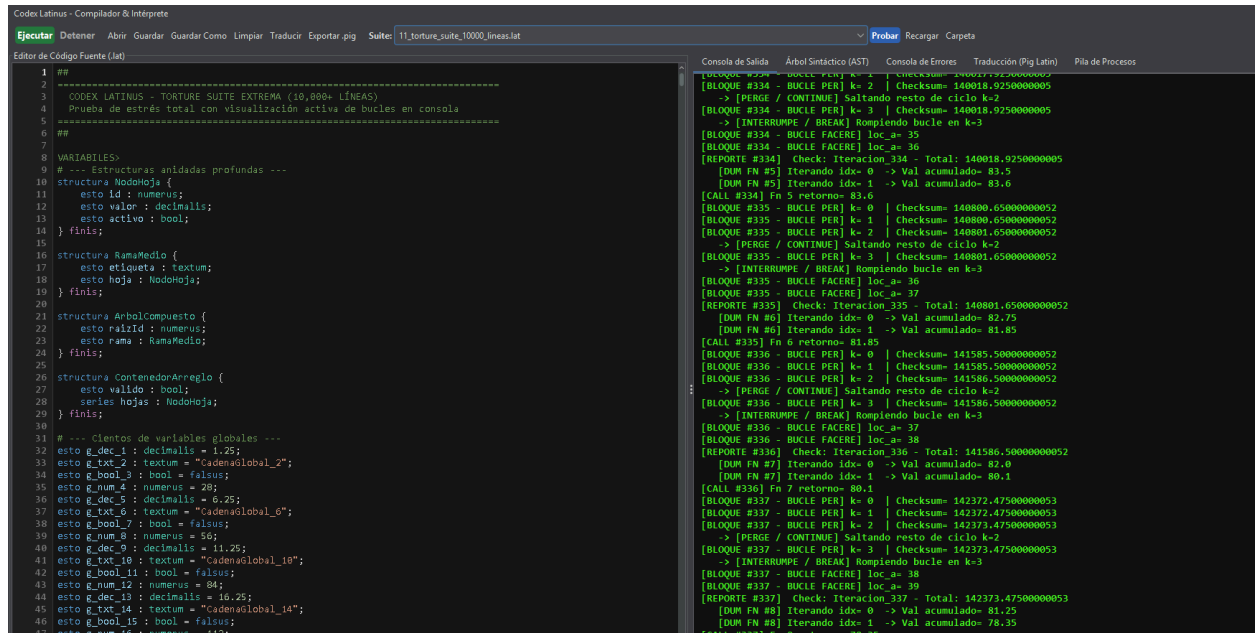
1. Inicio de la Aplicación



Esta sección describe la interfaz central de la herramienta donde el usuario interactúa con el código.

- **1.1. Ejecutar:** Esta función compila e interpreta el código fuente escrito en el panel izquierdo (.lat). Permite al usuario iniciar la ejecución del programa y ver el resultado procesado en el panel de la derecha, mostrando la salida, lógica o los resultados de las operaciones programadas.
- **1.2. Detener:** Esta opción permite al usuario interrumpir de forma inmediata la ejecución de un programa en curso. Es útil cuando se están probando bucles o procesos largos y se requiere cancelar la operación sin cerrar la aplicación.

1.1. Ejecutar



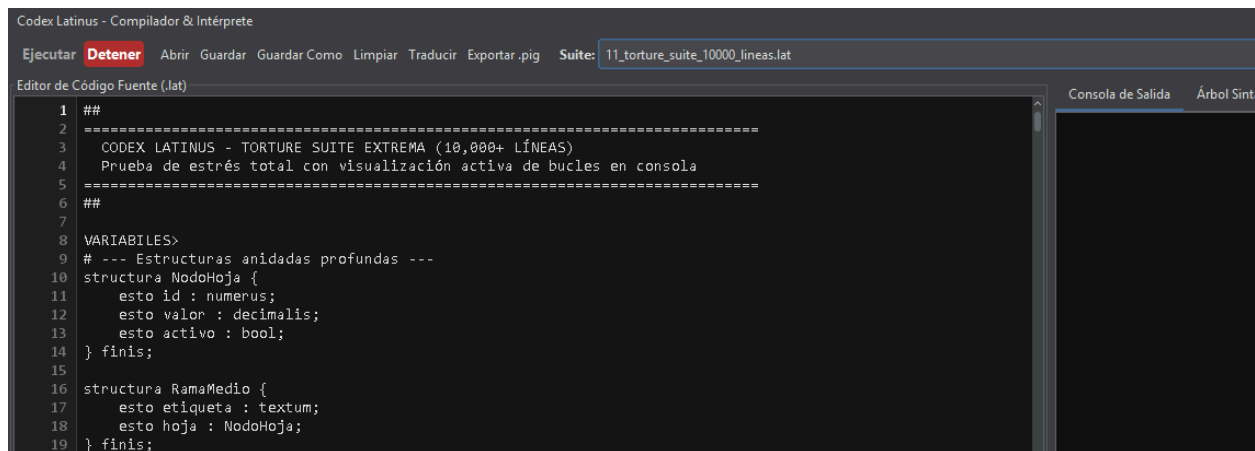
The screenshot shows the Codex Latinus IDE with the 'Ejecutar' (Execute) button highlighted. The source code in the left pane is a Latin program named '11_torture_suite_10000_lineas.lat'. The right pane displays the execution console, showing a detailed log of the program's execution, including variable values, loop iterations, and checksums.

```
1 ##
2 =====
3 CODEX LATINUS - TORTURE SUITE EXTREMA (10,000+ LÍNEAS)
4 Prueba de estrés total con visualización activa de bucles en consola
5 =====
6 ##
7
8 VARIABLES>
9 # --- Estructuras anidadas profundas ---
10 structura NodoHoja {
11     esto id : numerus;
12     esto valor : decimalis;
13     esto activo : bool;
14 } finis;
15
16 structura RamaMedio {
17     esto etiqueta : textum;
18     esto hoja : NodoHoja;
19 } finis;
20
21 structura ArbolCompuesto {
22     esto ralizid : numerus;
23     esto rama : RamaMedio;
24 } finis;
25
26 structura ContenedorArreglo {
27     esto valido : bool;
28     series hojas : NodoHoja;
29 } finis;
30
31 # --- Clientos de variables globales ---
32 esto g_dec_1 : decimalis = 1.25;
33 esto g_txt_2 : textum = "CadenaGlobal_2";
34 esto g_bool_3 : bool = falsus;
35 esto g_num_4 : numerus = 28;
36 esto g_dec_5 : decimalis = 0.25;
37 esto g_txt_6 : textum = "CadenaGlobal_6";
38 esto g_bool_7 : bool = falsus;
39 esto g_num_8 : numerus = 56;
40 esto g_dec_9 : decimalis = 11.25;
41 esto g_txt_10 : textum = "CadenaGlobal_10";
42 esto g_bool_11 : bool = falsus;
43 esto g_num_12 : numerus = 94;
44 esto g_dec_13 : decimalis = 10.25;
45 esto g_txt_14 : textum = "CadenaGlobal_14";
46 esto g_bool_15 : bool = falsus;
47 esto g_num_16 : numerus = 112;
```

The execution console on the right shows the following output:

```
[BLOQUE #334 - BUCLE PER] k= 2 | Checksum= 140018.92500000005
-> [PERGE / CONTINUE] Saltando resto de ciclo k=2
[BLOQUE #334 - BUCLE PER] k= 3 | Checksum= 140018.92500000005
-> [INTERROMPE / BREAK] Rompiendo bucle en k=3
[BLOQUE #334 - BUCLE FACERE] loc_a= 35
[BLOQUE #334 - BUCLE FACERE] loc_a= 36
[REPORTE #334] Check: Iteracion_334 - Total: 140018.92500000005
[DMX FN #5] Iterando idx= 0 -> Val acumulado= 83.5
[DMX FN #5] Iterando idx= 1 -> Val acumulado= 83.6
[CALL #334] Fn 5 retorno= 83.6
[BLOQUE #335 - BUCLE PER] k= 0 | Checksum= 140800.650000000052
[BLOQUE #335 - BUCLE PER] k= 1 | Checksum= 140800.650000000052
[BLOQUE #335 - BUCLE PER] k= 2 | Checksum= 140801.650000000052
-> [PERGE / CONTINUE] Saltando resto de ciclo k=2
[BLOQUE #335 - BUCLE PER] k= 3 | Checksum= 140801.650000000052
-> [INTERROMPE / BREAK] Rompiendo bucle en k=3
[BLOQUE #335 - BUCLE FACERE] loc_a= 36
[BLOQUE #335 - BUCLE FACERE] loc_a= 37
[REPORTE #335] Check: Iteracion_335 - Total: 140801.650000000052
[DMX FN #6] Iterando idx= 0 -> Val acumulado= 82.75
[DMX FN #6] Iterando idx= 1 -> Val acumulado= 81.85
[CALL #335] Fn 6 retorno= 81.85
[BLOQUE #336 - BUCLE PER] k= 0 | Checksum= 141585.500000000052
[BLOQUE #336 - BUCLE PER] k= 1 | Checksum= 141585.500000000052
[BLOQUE #336 - BUCLE PER] k= 2 | Checksum= 141586.500000000052
-> [PERGE / CONTINUE] Saltando resto de ciclo k=2
[BLOQUE #336 - BUCLE PER] k= 3 | Checksum= 141586.500000000052
-> [INTERROMPE / BREAK] Rompiendo bucle en k=3
[BLOQUE #336 - BUCLE FACERE] loc_a= 37
[BLOQUE #336 - BUCLE FACERE] loc_a= 38
[REPORTE #336] Check: Iteracion_336 - Total: 141586.500000000052
[DMX FN #7] Iterando idx= 0 -> Val acumulado= 82.0
[DMX FN #7] Iterando idx= 1 -> Val acumulado= 80.1
[CALL #336] Fn 7 retorno= 80.1
[BLOQUE #337 - BUCLE PER] k= 0 | Checksum= 142372.475000000053
[BLOQUE #337 - BUCLE PER] k= 1 | Checksum= 142372.475000000053
[BLOQUE #337 - BUCLE PER] k= 2 | Checksum= 142373.475000000053
-> [PERGE / CONTINUE] Saltando resto de ciclo k=2
[BLOQUE #337 - BUCLE PER] k= 3 | Checksum= 142373.475000000053
-> [INTERROMPE / BREAK] Rompiendo bucle en k=3
[BLOQUE #337 - BUCLE FACERE] loc_a= 38
[BLOQUE #337 - BUCLE FACERE] loc_a= 39
[REPORTE #337] Check: Iteracion_337 - Total: 142373.475000000053
[DMX FN #8] Iterando idx= 0 -> Val acumulado= 81.25
[DMX FN #8] Iterando idx= 1 -> Val acumulado= 78.35
[CALL #337] Fn 8 retorno= 78.35
```

1.2. Detener



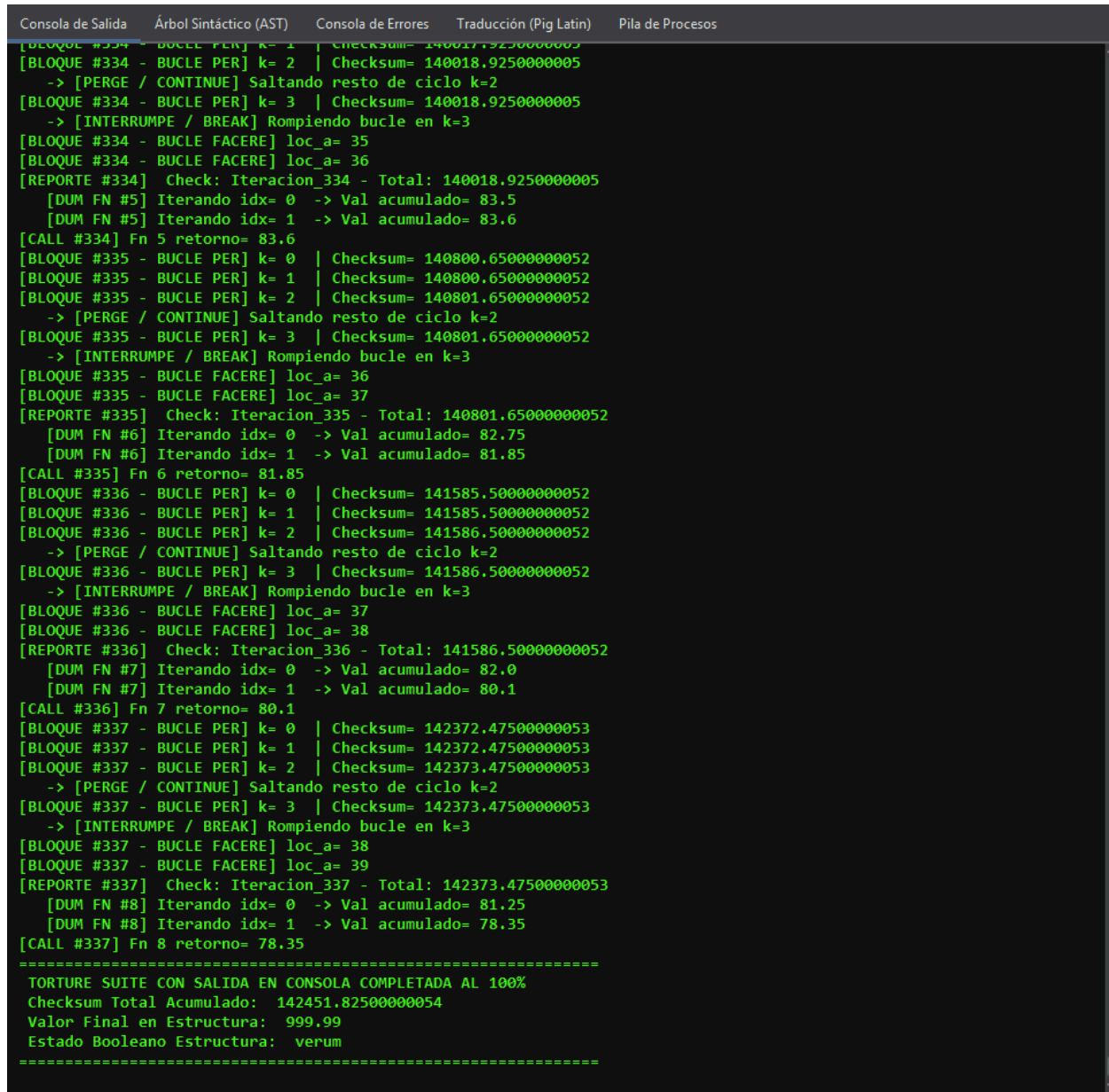
The screenshot shows the Codex Latinus IDE with the 'Detener' (Stop) button highlighted. The source code in the left pane is the same Latin program as in the previous screenshot. The right pane shows the 'Consola de Salida' (Output Console) and 'Árbol Sintáctico' (Syntax Tree) tabs, which are currently empty.

```
1 ##
2 =====
3 CODEX LATINUS - TORTURE SUITE EXTREMA (10,000+ LÍNEAS)
4 Prueba de estrés total con visualización activa de bucles en consola
5 =====
6 ##
7
8 VARIABLES>
9 # --- Estructuras anidadas profundas ---
10 structura NodoHoja {
11     esto id : numerus;
12     esto valor : decimalis;
13     esto activo : bool;
14 } finis;
15
16 structura RamaMedio {
17     esto etiqueta : textum;
18     esto hoja : NodoHoja;
19 } finis;
```

2. Apartados de la aplicación

2.1. Consola

Esta área permite al usuario depurar y analizar el comportamiento del código a través de diferentes vistas especializadas:

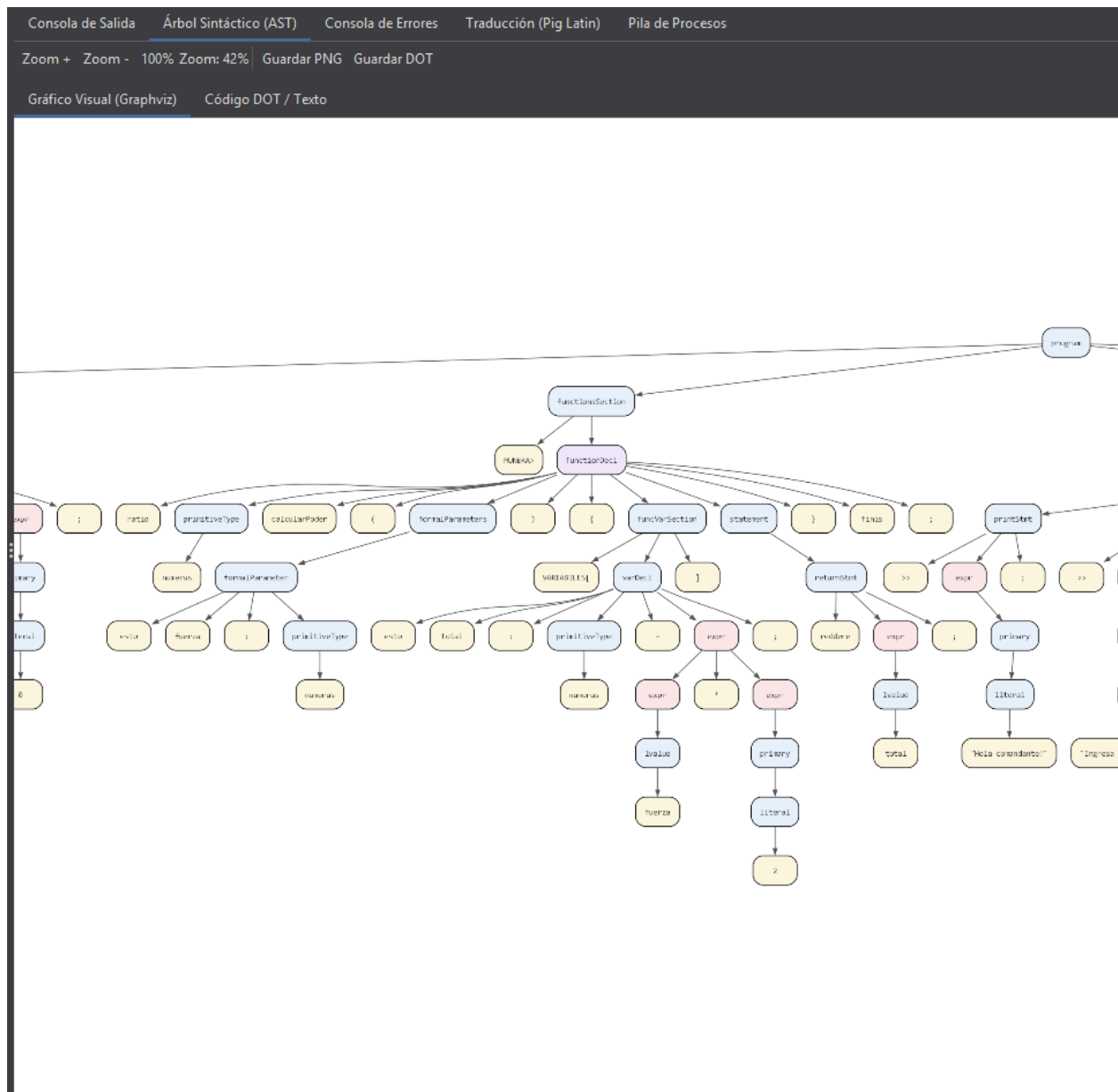


The screenshot displays the 'Consola de Salida' (Output Console) of the Torture Suite application. The console window has a dark background with green text. At the top, there are five tabs: 'Consola de Salida', 'Árbol Sintáctico (AST)', 'Consola de Errores', 'Traducción (Pig Latin)', and 'Pila de Procesos'. The 'Consola de Salida' tab is active, showing a detailed log of program execution. The log includes block numbers (e.g., [BLOQUE #334]), loop types (e.g., BUCLE PER, BUCLE FACERE), loop counts (k=2, k=3), checksums, and function calls (e.g., [CALL #334] Fn 5 retorno= 83.6). It also shows iteration details for various functions (e.g., [DUM FN #5] Iterando idx= 0 -> Val acumulado= 83.5). The log concludes with a summary: 'TORTURE SUITE CON SALIDA EN CONSOLA COMPLETADA AL 100%', 'Checksum Total Acumulado: 142451.82500000054', 'Valor Final en Estructura: 999.99', and 'Estado Booleano Estructura: verum'.

```
[BLOQUE #334 - BUCLE PER] k= 1 | Checksum= 140017.9250000005
[BLOQUE #334 - BUCLE PER] k= 2 | Checksum= 140018.9250000005
-> [PERGE / CONTINUE] Saltando resto de ciclo k=2
[BLOQUE #334 - BUCLE PER] k= 3 | Checksum= 140018.9250000005
-> [INTERRUMPE / BREAK] Rompiendo bucle en k=3
[BLOQUE #334 - BUCLE FACERE] loc_a= 35
[BLOQUE #334 - BUCLE FACERE] loc_a= 36
[REPORTE #334] Check: Iteracion_334 - Total: 140018.9250000005
[DUM FN #5] Iterando idx= 0 -> Val acumulado= 83.5
[DUM FN #5] Iterando idx= 1 -> Val acumulado= 83.6
[CALL #334] Fn 5 retorno= 83.6
[BLOQUE #335 - BUCLE PER] k= 0 | Checksum= 140800.65000000052
[BLOQUE #335 - BUCLE PER] k= 1 | Checksum= 140800.65000000052
[BLOQUE #335 - BUCLE PER] k= 2 | Checksum= 140801.65000000052
-> [PERGE / CONTINUE] Saltando resto de ciclo k=2
[BLOQUE #335 - BUCLE PER] k= 3 | Checksum= 140801.65000000052
-> [INTERRUMPE / BREAK] Rompiendo bucle en k=3
[BLOQUE #335 - BUCLE FACERE] loc_a= 36
[BLOQUE #335 - BUCLE FACERE] loc_a= 37
[REPORTE #335] Check: Iteracion_335 - Total: 140801.65000000052
[DUM FN #6] Iterando idx= 0 -> Val acumulado= 82.75
[DUM FN #6] Iterando idx= 1 -> Val acumulado= 81.85
[CALL #335] Fn 6 retorno= 81.85
[BLOQUE #336 - BUCLE PER] k= 0 | Checksum= 141585.50000000052
[BLOQUE #336 - BUCLE PER] k= 1 | Checksum= 141585.50000000052
[BLOQUE #336 - BUCLE PER] k= 2 | Checksum= 141586.50000000052
-> [PERGE / CONTINUE] Saltando resto de ciclo k=2
[BLOQUE #336 - BUCLE PER] k= 3 | Checksum= 141586.50000000052
-> [INTERRUMPE / BREAK] Rompiendo bucle en k=3
[BLOQUE #336 - BUCLE FACERE] loc_a= 37
[BLOQUE #336 - BUCLE FACERE] loc_a= 38
[REPORTE #336] Check: Iteracion_336 - Total: 141586.50000000052
[DUM FN #7] Iterando idx= 0 -> Val acumulado= 82.0
[DUM FN #7] Iterando idx= 1 -> Val acumulado= 80.1
[CALL #336] Fn 7 retorno= 80.1
[BLOQUE #337 - BUCLE PER] k= 0 | Checksum= 142372.47500000053
[BLOQUE #337 - BUCLE PER] k= 1 | Checksum= 142372.47500000053
[BLOQUE #337 - BUCLE PER] k= 2 | Checksum= 142373.47500000053
-> [PERGE / CONTINUE] Saltando resto de ciclo k=2
[BLOQUE #337 - BUCLE PER] k= 3 | Checksum= 142373.47500000053
-> [INTERRUMPE / BREAK] Rompiendo bucle en k=3
[BLOQUE #337 - BUCLE FACERE] loc_a= 38
[BLOQUE #337 - BUCLE FACERE] loc_a= 39
[REPORTE #337] Check: Iteracion_337 - Total: 142373.47500000053
[DUM FN #8] Iterando idx= 0 -> Val acumulado= 81.25
[DUM FN #8] Iterando idx= 1 -> Val acumulado= 78.35
[CALL #337] Fn 8 retorno= 78.35
=====
TORTURE SUITE CON SALIDA EN CONSOLA COMPLETADA AL 100%
Checksum Total Acumulado: 142451.82500000054
Valor Final en Estructura: 999.99
Estado Booleano Estructura: verum
=====
```

- **2.1. Consola:** Muestra la salida estándar del programa después de la ejecución. Aquí es donde el usuario puede visualizar los resultados, impresiones (print) y el flujo de los procesos en tiempo real.

2.2. Árbol Sintáctico



- **2.2. Árbol Sintáctico (AST):** Proporciona una representación visual de la estructura jerárquica del código fuente analizado. Ayuda al usuario a entender cómo el compilador descompone la lógica y las estructuras del lenguaje.

2.3. Consola de errores

Consola de Salida	Árbol Sintáctico (AST)	Consola de Errores	Traducción (Pig Latin)	Pila de Procesos
TIPO	LÍNEA	COLUMNA	DESCRIPCIÓN	
SEMANTICO	87	36	Operación incompatible: textum no admite resta.	
SEMANTICO	87	1	Tipo incompatible: No se puede inicializar la variable 'error_resta_texto' de tipo 'numerus' con valo	
SEMANTICO	88	35	Operación incompatible: textum solo admite concatenación con '+'.	
SEMANTICO	88	1	Tipo incompatible: No se puede inicializar la variable 'error_mult_texto' de tipo 'numerus' con valor	
SEMANTICO	91	36	Operación incompatible: No se permite aritmética con tipos booleanos.	
SEMANTICO	92	33	Operación incompatible: No se permite aritmética con tipos booleanos.	
SEMANTICO	95	33	Error Aritmético: División entre cero.	
SEMANTICO	96	33	Error Aritmético: Módulo entre cero.	
SEMANTICO	99	1	Corrupción de Flujo: La condición de la sentencia 'si' debe ser estrictamente de tipo booleano.	
SEMANTICO	103	1	Corrupción de Flujo: La condición del ciclo 'dum' debe ser estrictamente de tipo booleano.	
SEMANTICO	108	1	Variable no declarada: 'variable_inexistente'.	
SEMANTICO	111	33	Índice fuera de límites: 99	
SEMANTICO	112	1	Índice fuera de límites: 50	
SEMANTICO	115	1	Atributo 'atributo_falso' no existe en la estructura.	

- **2.3. Consola de Errores:** Lista detalladamente cualquier error detectado durante la compilación (semánticos, de sintaxis, etc.). Indica al usuario el tipo de error, la línea y la columna donde ocurre, facilitando la corrección rápida del código.

2.4. Traducción Pig Latin

Consola de Salida	Árbol Sintáctico (AST)	Consola de Errores	Traducción (Pig Latin)	Pila de Procesos
<pre>ARIABLESVAY[estoway esultadoray : ecimalisday = apitalcay;] erpay (estoway iway : umerusnay = 0; iway < anosway; iway++) { esultadoray = esultadoray + esultadoray * asatay; } inisfay; eddereray esultadoray; } inisfay; actloway ostrarResumenTiposmay() { %OINK %OINK "Entero: " %OINK entero_valway; %OINK %OINK "Decimal: " %OINK ecimal_valday; %OINK %OINK "Texto: " %OINK exto_valtay; %OINK %OINK "Caracter: " %OINK aracter_valcay; %OINK %OINK "Booleano V: " %OINK ooleano_vbay %OINK " Booleano F: " %OINK ooleano_fbay; %OINK %OINK "Booleano Bool: " %OINK ooleano_tipadolbay %OINK " Booleano Boolean: " %OINK ooleano_tipado2bay; } inisfay; AIORMAY; %OINK %OINK "====="; %OINK %OINK ">>> 1. OPERACIONES VALIDAS Y PROMOCION DE TIPOS"; %OINK %OINK "====="; ostrarResumenTiposmay(); estoway una_mixtasay : ecimalisday = entero_valway + ecimal_valday; %OINK %OINK "Entero (42) + Decimal (3.14159) = " %OINK una_mixtasay; estoway oncat1cay : extumtay = "Saldo de " + i_cuentamay.itulartay + "; \$" + i_cuentamay.aldosay; estoway oncat2cay : extumtay = "Estado cuenta bloqueada: " + i_cuentamay.oqeada2bay; %OINK %OINK oncat1cay; %OINK %OINK oncat2cay; umeros_basenay[0] = umeros_basenay[0] * 2; umeros_basenay[4] = umeros_basenay[4] + 5; %OINK %OINK "Arreglo pos 0 modificado: " %OINK umeros_basenay[0]; %OINK %OINK "Arreglo pos 4 modificado: " %OINK umeros_basenay[4]; %OINK %OINK "Calculo de Interes Compuesto (1000 a 5% por 3 anos): " %OINK alcularInteresnay(1000.0, 0.05, 3); %OINK %OINK ""; %OINK %OINK "====="; %OINK %OINK ">>> 2. CASOS DE PRUEBA DISEÑADOS PARA PROVOCAR ERRORES SEMANTICOS"; %OINK %OINK " (El compilador debe capturar estos errores y listarlos en la tabla)"; %OINK %OINK "====="; estoway error_resta_textoway : umerusnay = "Hola" - 5; estoway error_mult_textoway : umerusnay = "Mundo" * 3; estoway error_mult_boolway : ecimalisday = erumway * 10.5; estoway error_div_boolway : umerusnay = alusfay / 2; estoway error_div_ceroway : umerusnay = 100 / 0; estoway error_mod_ceroway : umerusnay = 50 % 0; isay (100 + 50) { %OINK %OINK "Este bloque NO debe ejecutarse por corrupcion de flujo"; } inisfay; umday ("cadena_no_booleana") { %OINK %OINK "Bucle invalido por corrupcion de flujo"; } inisfay; ariable_inexistentevey = 9999; estoway uera_de_rangofay : umerusnay = umeros_basenay[99]; umeros_basenay[50] = 1234; i_cuentamay.atributo_falsoway = 100; %OINK %OINK ""; %OINK %OINK "=== PRUEBA DE TIPOS Y ERRORES SEMANTICOS FINALIZADA ==="; inisfay;</pre>				

- **2.4. Traducción (Pig Latin):** Muestra el código fuente convertido a una sintaxis diferente (en este caso, un lenguaje llamado "Pig Latin"). Útil para verificar transformaciones de código o lógica de traducción.

2.5. Pila de procesos

Consola de Salida Árbol Sintáctico (AST) Consola de Errores Traducción (Pig Latin) **Pila de Procesos**

Probar Recargar Carpeta Símbolos Tipos Errores

Inicio Anterior Siguiente Fin Reproducir

Paso: 441 / 948
Acción: reduce printStmt -> >> expr ;

Gráfica de la Pila (Bottom-Up)

- printStmt [TOP]
- statement
- MAIOR>
- functionsSection
- globalVarsSection

LOG de Operaciones Sintácticas

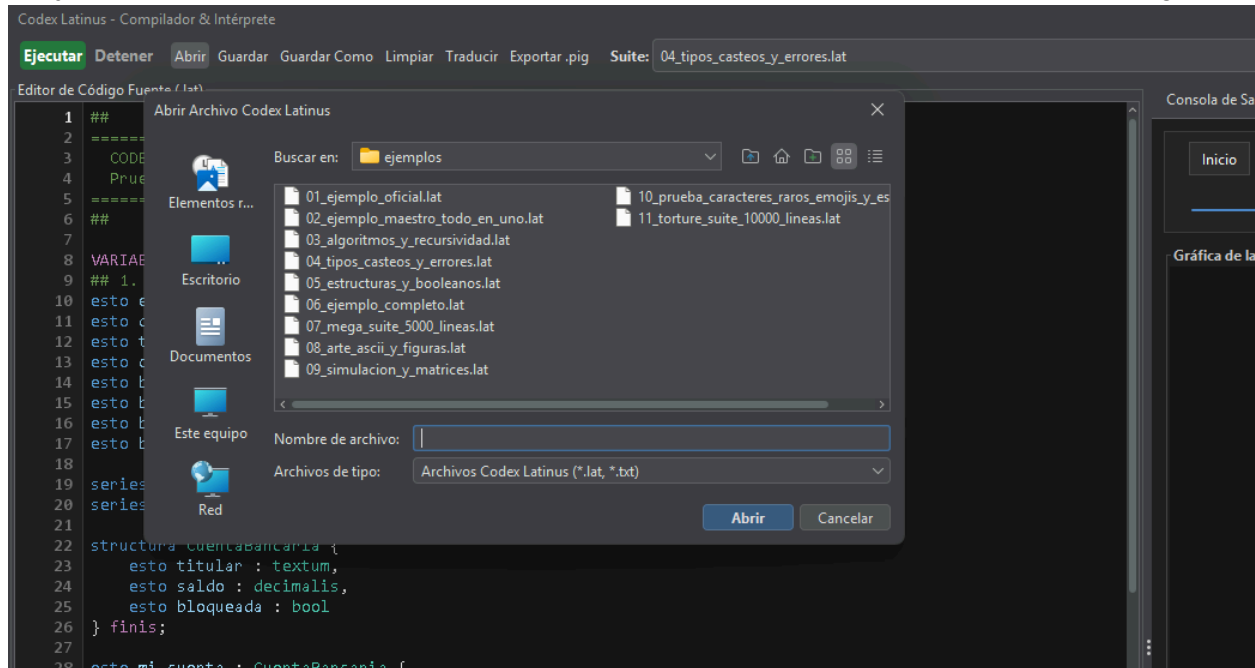
```
reduce expr -> lvalue
shift ;
reduce printStmt -> >> expr >> expr >> expr >> expr ;
reduce statement -> printStmt
shift >>
shift "Booleano Bool: "
reduce literal -> "Booleano Bool: "
reduce primary -> literal
reduce expr -> primary
shift >>
shift booleano_tipado1
reduce lvalue -> booleano_tipado1
reduce expr -> lvalue
shift >>
shift " | Booleano Boolean: "
reduce literal -> " | Booleano Boolean: "
reduce primary -> literal
reduce expr -> primary
shift >>
shift booleano_tipado2
reduce lvalue -> booleano_tipado2
reduce expr -> lvalue
shift ;
reduce printStmt -> >> expr >> expr >> expr >> expr ;
reduce statement -> printStmt
shift }
shift finis
shift ;
reduce functionDecl -> actio mostrarResumenTipos ( ) { statement statement
reduce functionsSection -> MUNERA> functionDecl functionDecl
shift MAIOR>
shift >>
shift "=====
reduce literal -> "=====
reduce primary -> literal
reduce expr -> primary
shift ;
reduce printStmt -> >> expr ;
reduce statement -> printStmt
shift >>
shift ">>> 1. OPERACIONES VALIDAS Y PROMOCION DE TIPOS"
reduce literal -> ">>> 1. OPERACIONES VALIDAS Y PROMOCION DE TIPOS"
reduce primary -> literal
reduce expr -> primary
shift ;
> reduce printStmt -> >> expr ;
```

- **2.5. Pila de Procesos:** Ofrece una vista paso a paso de las operaciones sintácticas realizadas por el compilador, permitiendo al usuario seguir el proceso de "reducción" y "desplazamiento" (bottom-up) de las instrucciones.

3. Barra de opciones

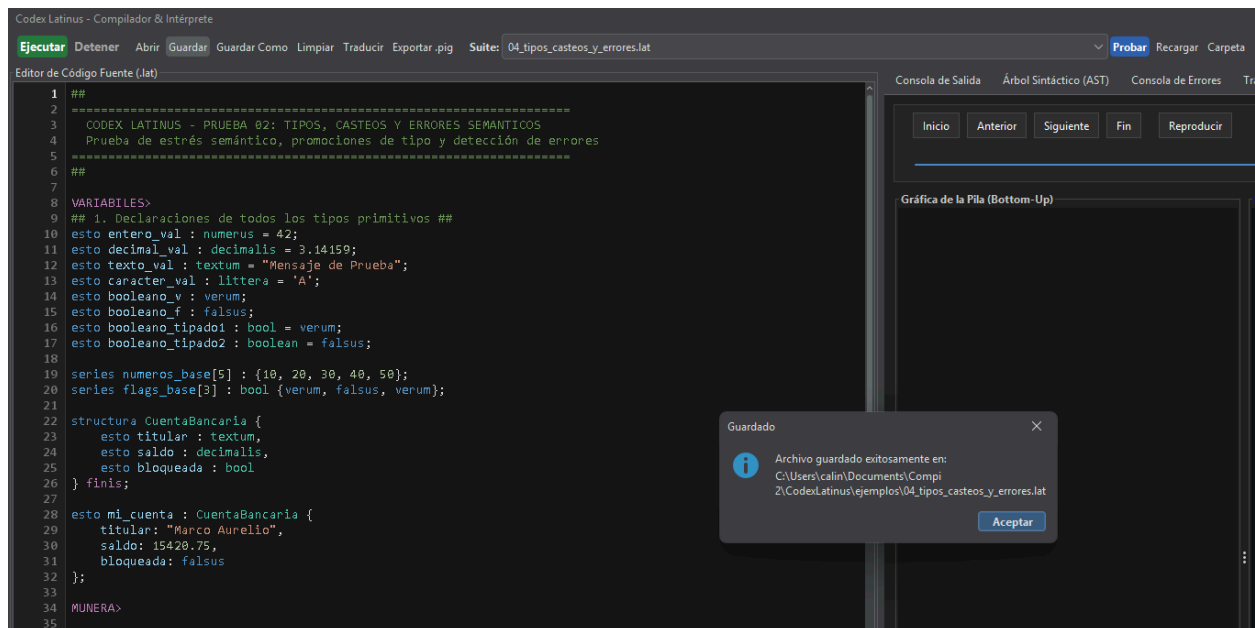
3.1. Abrir

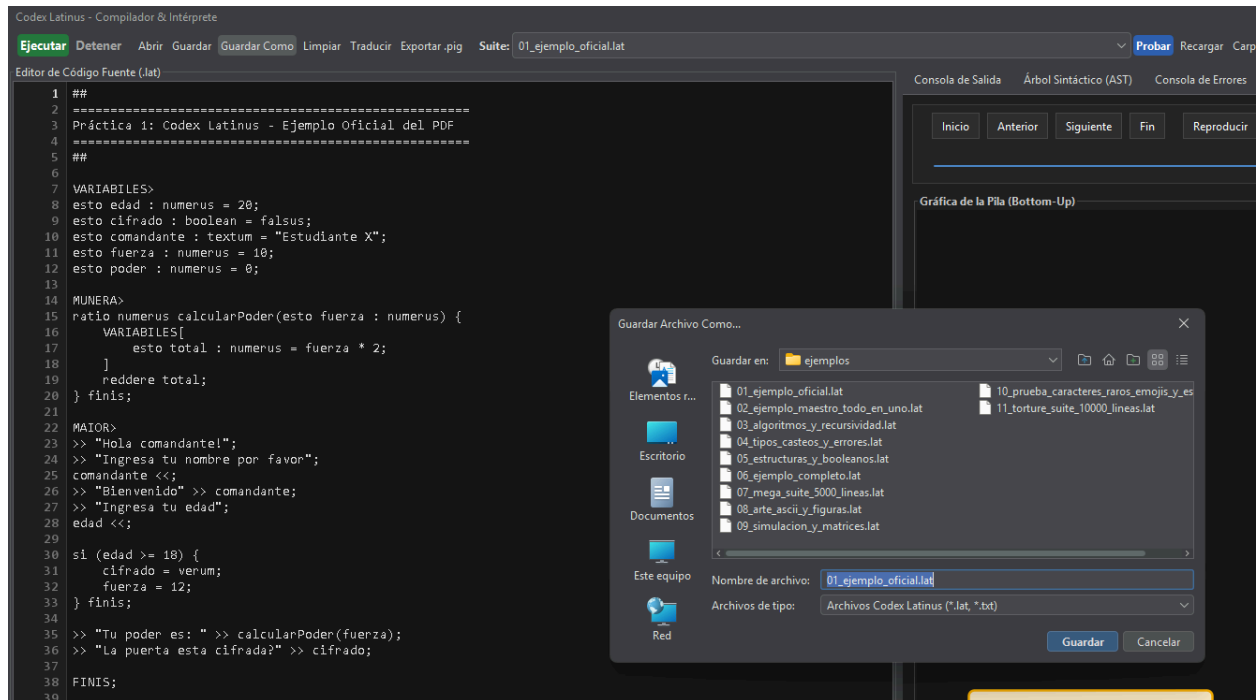
Conjunto de herramientas para la administración del ciclo de vida de los archivos de código:



- **3.1. Abrir:** Abre un selector de archivos nativo del sistema operativo, permitiendo al usuario buscar y cargar archivos con extensión .lat o .txt almacenados en su equipo.

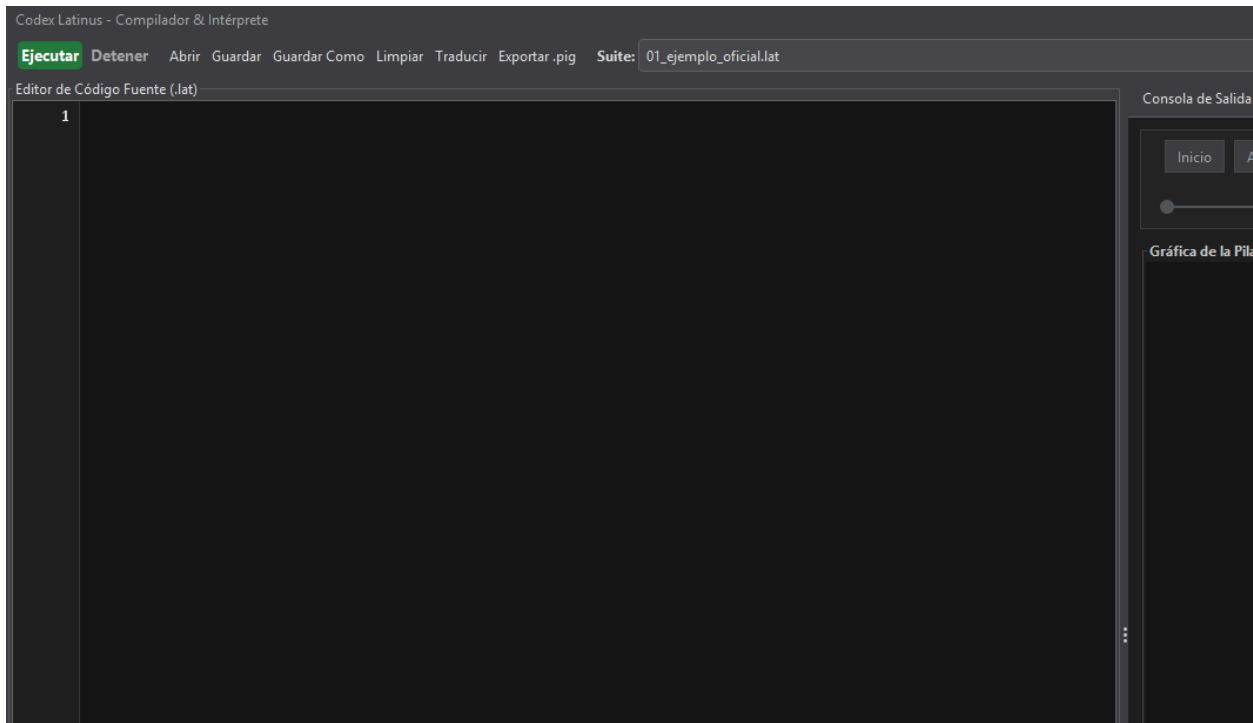
3.2. Guardar/Guardar Como





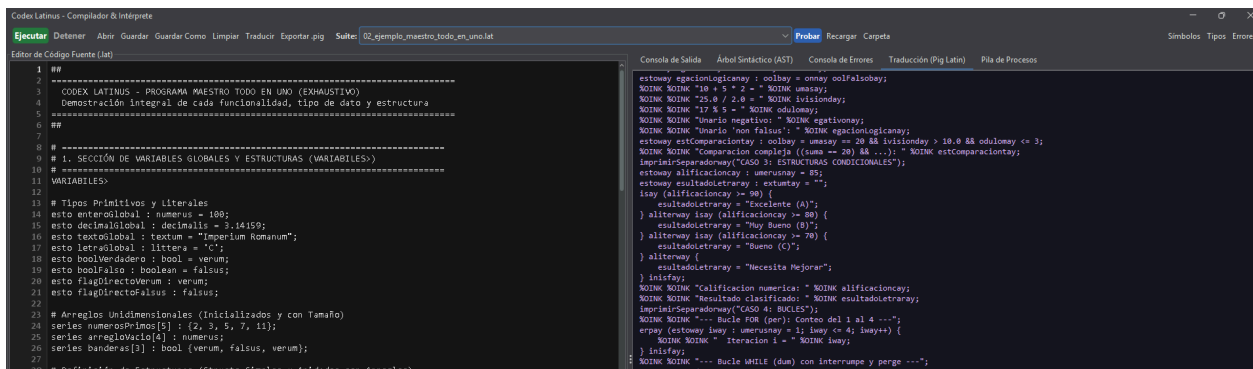
- **3.2. Guardar / Guardar Como:** Funcionalidades estándar para persistir el trabajo actual. "Guardar" sobrescribe el archivo abierto, mientras que "Guardar Como" permite exportar el código fuente a una nueva ubicación o con un nuevo nombre.

3.3. Limpiar



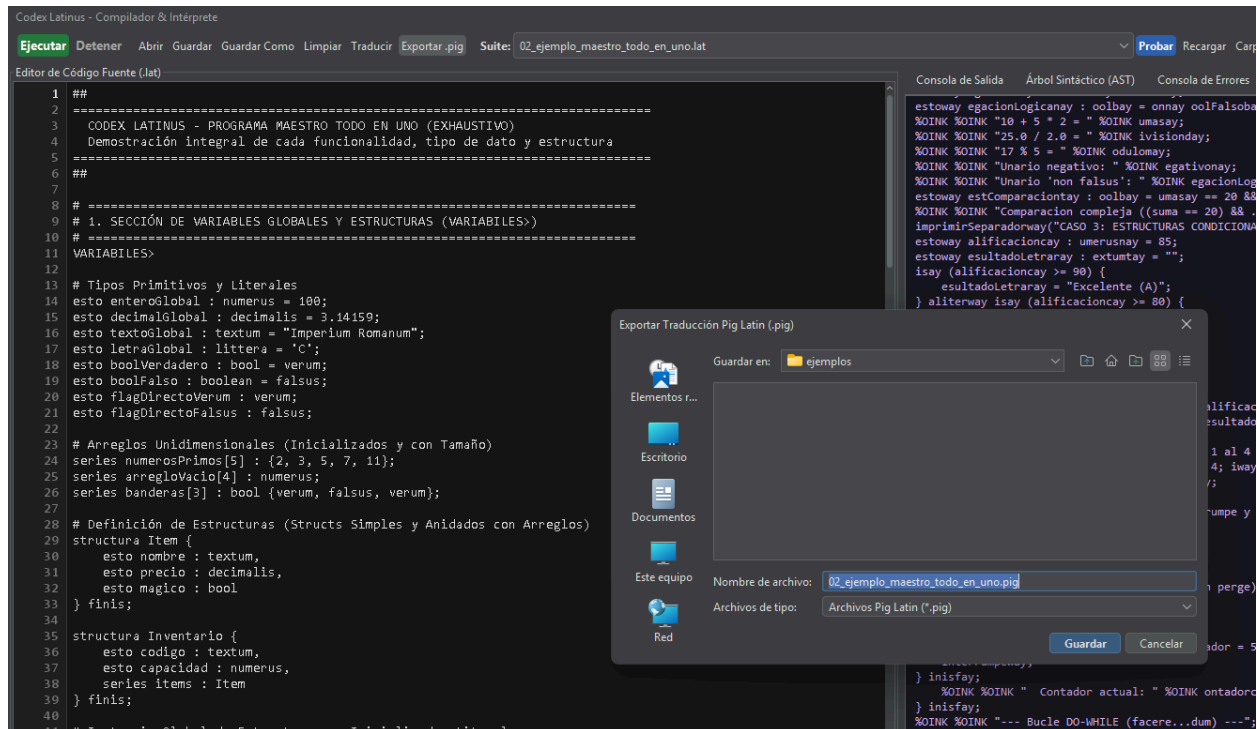
- **3.3. Limpiar:** Borra el contenido actual del editor de código fuente, dejando el área de trabajo en blanco para iniciar un nuevo proyecto o archivo desde cero.

3.4. Traducir



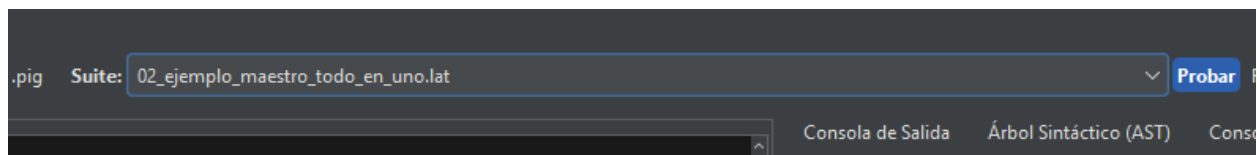
- **3.4. Traducir:** Ejecuta la función de conversión del código fuente al lenguaje destino (Pig Latin) y muestra el resultado en el panel correspondiente.

3.3. Exportar



- **3.5. Exportar (.pig):** Permite guardar el resultado de la traducción generada en un archivo físico (.pig) para su uso posterior o almacenamiento externo.

3.3. Suite



3.3. Probar/Recargar

The screenshot displays the Codex Latinus IDE interface. The top menu bar includes 'Ejecutar', 'Detener', 'Abrir', 'Guardar', 'Guardar Como', 'Limpiar', 'Traducir', 'Exportar .pig', and 'Suite: 01_ejemplo_oficial.lat'. The 'Probar' button is highlighted in the top right corner. The main editor shows the source code of a program, and the right-hand side displays the console output.

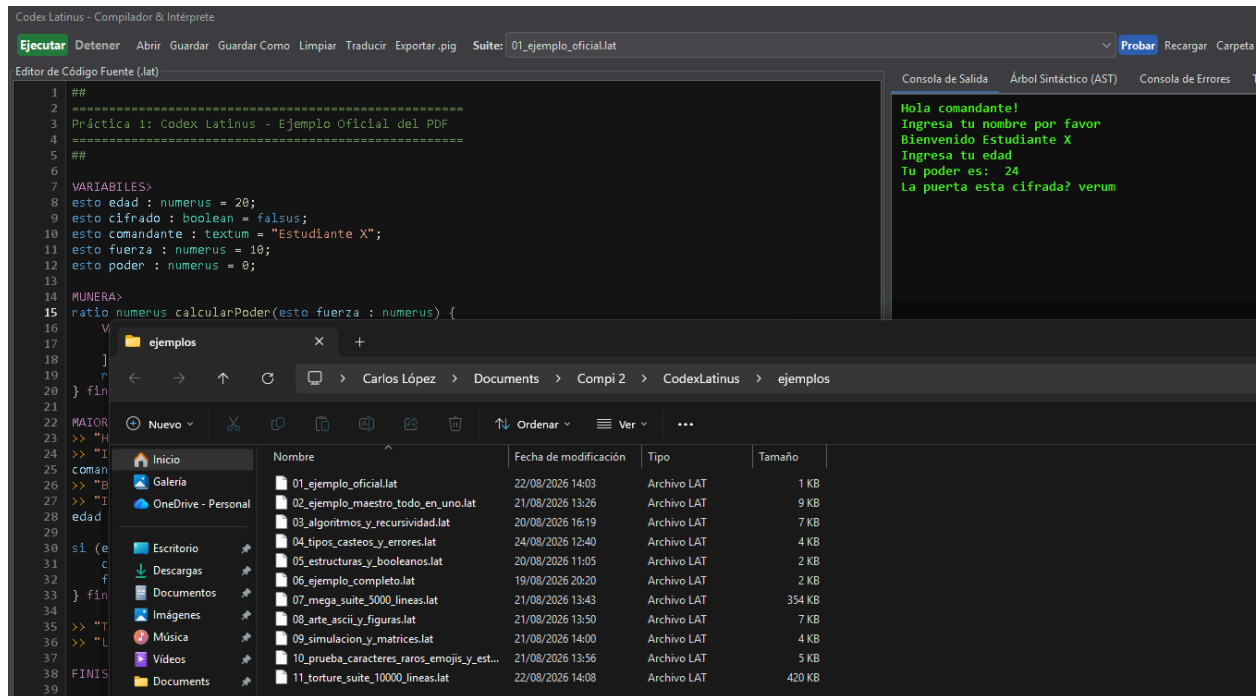
Source Code (Editor de Código Fuente (.lat)):

```
1 ##
2 =====
3 Práctica 1: Codex Latinus - Ejemplo Oficial del PDF
4 =====
5 ##
6
7 VARIABLES>
8 esto edad : numerus = 20;
9 esto cifrado : boolean = falsus;
10 esto comandante : textum = "Estudiante X";
11 esto fuerza : numerus = 10;
12 esto poder : numerus = 0;
13
14 MUNERA>
15 ratio numerus calcularPoder(esto fuerza : numerus) {
16     VARIABLES[
17         esto total : numerus = fuerza * 2;
18     ]
19     reddere total;
20 } finis;
21
22 MAIOR>
23 >> "Hola comandante!";
24 >> "Ingresa tu nombre por favor";
25 comandante <<;
26 >> "Bienvenido" >> comandante;
27 >> "Ingresa tu edad";
28 edad <<;
29
30 si (edad >= 18) {
31     cifrado = verum;
32     fuerza = 12;
33 } finis;
34
35 >> "Tu poder es: " >> calcularPoder(fuerza);
36 >> "La puerta esta cifrada?" >> cifrado;
37
38 FINIS;
39
```

Console Output (Consola de Salida):

```
Hola comandante!
Ingresa tu nombre por favor
Bienvenido Estudiante X
Ingresa tu edad
Tu poder es: 24
La puerta esta cifrada? verum
```

3.3. Carpeta



3.6. Carpeta: Abre el explorador de archivos del sistema en la ruta donde se encuentra el proyecto actual, facilitando al usuario el acceso directo a los archivos fuente y recursos asociados.